1.1 Big Data

Python for High Performace Data Analysis documentation

How large is your data?

How large is the data you are working with?

Are you experiencing performance bottlenecks when you try to analyse it?

“Big data refers to data sets that are too large or complex to be dealt with by traditional data-processing application software. […] Big data analysis challenges include capturing data, data storage, data analysis, search, sharing, transfer, visualization, querying, updating, information privacy, and data source.”

“Big dat unities are increasingly adopting high-throughput data production methods which lead to very large datasets. One driving force behind this development is the advent of powerful machine learning method he strong development of high performance comput hich can efficiently take advantage of the hardware.

This co ting which focuses on working with large data. The data can come from either computer models and simulations or from experiments and observations, and the goal is to preprocess, analyse and visualise it to generate scientific results.

1.2 Python

Performance bottlenecks in Python

Have you ever written Python scripts that look something like the one below?

Compared to C/C++/Fortran, this for-loop will probably be orders of magnitude slower

```
f = open("mydata.dat", "r")
for line in f.readlines():
    fields = line.split(",")
    x, y, z = fields[1], fields[2], fields[3]
    # some analysis with x, y and z
f.close()
```

Despite early design choices of the Python language which made it significantly slower than conventional HPC languages, a rich and growing ecosystem of open source libraries have established Python as an industry-standard programming language for working with data on all levels of the data analytics pipeline. These range from generic numerical libraries to special-purpose and/or domain-specific packages. This lesson is focused on introducing modern packages from the Python ecosystem to work with large data. Specifically, we will learn to use:

Numpy

Scipy

Pandas

Xarray

Numba

Cython

multithreading

multiprocessing

Dask

1.3 What You Will Learn

This lesson provides a broad overview of methods to work with large datasets using tools and libraries from the Python ecosystem. Since this field is fairly extensive we will not have time to go into much depth. Instead, the objective is to expose just enough details on each topic for you to get a good idea of the big picture and an understanding of what combination of tools and libraries will work well for your particular use case.

Python for High Performance Data Analysis

This (https://github.com/ENCCS/evita-material-template/blob/main/content/Python-HPDA.ipynb) links to the initial file for the lesson material template for the EVITA project.

⚙ Prerequisites

prerequisites

Callout

- Intended learning outcomes
- Connection to skills tree

Welcome to High Performance Data Analytics in Python!

Scientists, engineers, and professionals from many sectors are seeing an enormous growth in the size and number of datasets relevant to their domains. Professional titles have emerged to describe specialists working with data, such as data scientists and data engineers, but also other experts are finding it necessary to learn tools and techniques to work with big data. Typical tasks include preprocessing, analysing, modeling and visualising data.

This lesson will give an overview of working with research data in Python using general libraries for storing, processing, analysing and sharing data. The focus is on high performance. After covering tools for performant processing on single workstations the focus shifts to profiling and optimising, parallel and distributed computing.

Prerequisites

- Basic experience with Python
- Basic experience in working in a Linux-like terminal
- Some prior experience in working with large or small datasets

Software Setup

Setting up programming environment

Lesson Episodes:

1. Motivation
2. Efficient Array Computing

Learning Outcomes

By the end of a workshop covering this lesson, learners should:

- Have a good overview of available tools and libraries for improving performance in Python (***link to leaves in skill tree***)
- Knowing libraries for efficiently storing, reading and writing large data (***link to leaves in skill tree***)
- Be comfortable working with NumPy arrays and Pandas dataframes for data analysis using Python (***link to leaves in skill tree***)

Specifically, this course covers:

Tools for efficiently storing data and writing/reading data to/from disk

How to share datasets and mint digital object identifiers (DOI)

Main methods of efficiently working with tabular data and multidimensional arrays

How to measure performance and boost performance of time consuming Python functions

Various methods to parallelise Python code

1.4 Keypoints

Datasets are getting larger across nearly all scientific and engineering domains

The Python ecosystem has many libraries and packages for working with big data efficiently

Next Episode

2. Efficient Array Computing

This episode introduces how to write high-performance numerical code in Python packages (Numpy, Pandas, and Scipy) by leveraging tools and libraries designed to optimize computation speed and memory usage. It explores strategies such as vectorization with NumPy, just-in-time compilation using Numba, and parallelization techniques that can significantly reduce execution time. These methods help Python developers overcome the traditional performance limitations of the language, making it suitable for intensive scientific and engineering applications.

Contents of this notebook

Local Installation

Install miniforge

Install python programming environment on personal computer

Using HPC Cluster

Login to LUMI cluster

Running jobs on LUMI cluster

Learning objectives

Understand limitations of Python's standard library for large data processing

Understand the logic behind NumPy ndarrays and learn to use some NumPy numerical computing tools

Learn to use data structures and analysis tools from Panda

Instructor notes

25 min teaching/type-along

25 min exercising

Contents of this notebook

2.1 Why Can Python Be Slow?

2.2 Numpy

2.3 Pandas

2.4 Scipy

2.5 Numba

2.6 Quizzes

Local Installation

Install miniforge

If you already have a preferred way to manage Python versions and libraries, you can stick to that. If not, we recommend that you install Python3 and all libraries using Miniforge, a free minimal installer for the package, dependency and environment manager conda.

Please follow the installation instructions to install Miniforge.

Make sure that conda is correctly installed:

```
$ conda --version
# conda 24.11.2
```

Install python programming environment on personal computer

For Mac users

With Conda installed, install the required dependencies by running:

```
$ conda env create --yes -f https://raw.githubusercontent.com/ENCCS/hpda-python/main/content/environment.yml
```

This will create a new environment pyhpda which you need to activate by:

```
$ conda activate pyhpda
```

Ensure that the Python version is fairly recent:

2.1 Why Can Python Be Slow?

Computer programs are written in a language and then translated to the actual machine instructions that a processor understands. There are two main approaches for this translation:

For **compiled** programming languages, the translation is done by a compiler before the execution of the program

For **interpreted** languages, the translation is done by an interpreter during the execution of the program

Credits

Don't forget to check out additional **course materials** from XXX. Please **contact us** if you want to reuse these course materials in your teaching. You can also join the **XXX channel** to share your experience and get more help from the community.

Licensing

Copyright © 2025 XXX. This material is released by XXX under the Creative Commons Attribution 4.0 International (CC BY 4.0). These materials may include references to hardware and software developed by other entities; all applicable licensing and copyrights apply.

Compiled languages are typically more efficient, but the behaviour of the program during runtime is more static than with interpreted languages. The development as with interpreted languages.

```
$ python --version
# Python 3.12.8
```

Python is an interpreted language, and many features that make development rapid with Python are a result of that, with the price of reduced performance in many cases.

```
$ jupyter-lab
```

2.1.1 Dynamic typing

Python is a dynamic language. Variables get a type only during the runtime when values (Python objects) are assigned to them, so it is more difficult for the interpreter to optimize the execution. In comparison, a compiler can make extensive analysis and optimization before the execution. Even though there has in recent years been a lot of progress in just-in-time (**JIT**) compilation techniques that allow programs to be optimized at runtime, the inherent, dynamic nature of the Python programming language remains one of its main performance bottlenecks.

2.1.2 Flexible data structures

The built-in data structures of Python, such as lists and dictionaries, are very flexible, but they are also very generic which makes them not well suited for extensive numerical computations. Even though the implementation of data structures is often quite efficient when processing different types of data, there is a lot of overhead due to the generic nature of these data structures when processing only a single type of data.

In summary, the flexibility and dynamic nature of Python, which enhances programmer productivity greatly, is also the main cause for the performance problems. Fortunately, as we discuss in the course, many of the bottlenecks can be circumvented.

2.2 NumPy

As probably the most fundamental building block of the scientific computing ecosystem in Python, NumPy offers comprehensive mathematical functions, random number generators, linear algebra routines, Fourier transforms, and more.

NumPy is based on well-optimized C code, which gives much better performace than regular Python. In particular, by using homogeneous data structures, NumPy *vectorizes* mathematical operations where fast pre-compiled code can be applied to a sequence of data instead of using traditional `for` loops.

2.2.1 Arrays

ndarrays can have multiple dimensions, *e.g.* a 1-D array is a vector, a 2-D array is a matrix

ndarrays are fast only when all data elements are of the same type

```
import numpy as np

# create float32 variable
x = np.float32(1.0)

# array with uint8 unsigned integers
z = np.arange(3, dtype=np.uint8)

# convert array to floats
z.astype(float)
```

2.2.3 Creating NumPy arrays

One way to create a NumPy array is to convert from a Python list, but make sure that the list is homogeneous (contains same data type) otherwise performace will be downgraded. Since appending elements to an existing array is slow, it is a common practice to preallocate the necessary space with `np.zeros` or `np.empty` when converting from a Python list is not possible.

Using HPC Cluster

Login to LUMI cluster

Follow practical instructions **HERE** to get your access to LUMI cluster.

On Step 5, you can login to LUMI cluster through terminal.

On Step 6, you can login to LUMI cluster from the web-interface.

Running jobs on LUMI cluster

If you want to run an interactive job asking for 1 node, 1 GPU, and 1 hour:

```
$ salloc -A project_XXXXX -N 1 -t 1:00:00 -p standard-g --gpus-per-node=1

$ srun <some-command>
```

Exit interactive allocation with `exit`

You can also submit your job with a batch script submit.sh:

```
#!/bin/bash -l
#SBATCH --account=project_XXXXX
#SBATCH --job-name=example-job
#SBATCH --output=examplejob.o%j
#SBATCH --error=examplejob.e%j
#SBATCH --partition=standard-g
#SBATCH --nodes=1
#SBATCH --gpus-per-node=1
#SBATCH --ntasks-per-node=1
#SBATCH --time=1:00:00

srun <some_command>
```

Using `pyhpda` programming environment

We have installed the `pyhpda` programming environment on LUMI. You can follow instructions below to activate it and login to LUMI cluster, either via terminal or through the web-interface.

If you use VS code, you can come to the installed `pyhpda` programming environment via choosing `Select Kernel` ar the upper right corner, `Python Environments` and you will find the pre-installed `pyhpda` programming environment.

For Linux users

Please provide detailes instructrs for linux users to install packages.

For Windows users

Please provide detailes instructrs for windows users to install packages.

```
import numpy as np

a = np.array((1, 2, 3, 4), float)
print(f"a = {a}\n")
# array([ 1., 2., 3., 4.])

list1 = [[1, 2, 3], [4, 5, 6]]
mat = np.array(list1, complex)
# create complex array, with imaginary part equal to zero
print(f"mat = \n {mat} \n")
# array([[ 1.+0.j, 2.+0.j, 3.+0.j],
#        [ 4.+0.j, 5.+0.j, 6.+0.j]])

print(f"mat.shape={mat.shape}, mat.size={mat.size}")
# mat.shape=(2, 3), mat.size=6
# you should get output as shown below
/project/project_XXXXX/miniconda3/envs/pyhpda/bin/python
 conda
```

```
a = np.arange(10)
print(a)
# array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

b = np.arange(0.1, 0.2, 0.02)
print(b)
# array([0.1 , 0.12, 0.14, 0.16, 0.18])

c = np.linspace(-4.5, 4.5, 5)
print(c)
# array([-4.5 , -2.25, 0. , 2.25, 4.5 ])
```

Time: `4:00:00`

Array with given shape initialized to `zeros`, `ones`, arbitrary value (`full`) or unitialized (`empty`):

Working directory:

```
a = np.zeros((4, 6), float)
print(a.shape)
# (4, 6)

b = np.ones((2, 4))
print(b)
# array([[ 1., 1., 1., 1.],
#        [ 1., 1., 1., 1.]])

c = np.full((2, 3), 4.2)
print(c)
# array([[4.2, 4.2, 4.2],
#        [4.2, 4.2, 4.2]])

d = np.empty([3, 3])
print(d)
# array([[0.0000e+000 0.0000e+000 0.0000e+000]
#        [0.0000e+000 0.0000e+000 1.3597e-320]
#        [0.0000e+000 0.0000e+000 0.0000e+000]]
```

This episode provides a broad overview of this course and the main motivation to attend this course.

Similar arrays as an existing array:

```
a = np.zeros((4, 6), float)
b = np.empty_like(a)
c = np.ones_like(a)
d = np.full_like(a, 9.1)
print(f"a={a}\n\n b={b}\n\n c={c}\n\n d={d}")
```

What is the Python programming environment and the ecosystem

2.2.4 Array operations and manipulations

What you will learn during this course

All the familiar arithmetic operators in NumPy are applied elementwise:

```
# 1D example

import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(f" a+b = {a+b}\n a/b = {a/b}")
```

Contents of this notebook

1.1 Big Data

1.2 Python

```
# 2D example

import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
b = np.array([[10, 10, 10], [10, 10, 10]])

print(a+b)
# [[11, 12, 13],
#  [14, 15, 16]]
```

2.2.5 Array indexing

Basic indexing is similar to Python lists. Note that advanced indexing creates copies of arrays.

```
# 1D example

import numpy as np

data = np.array([1,2,3,4,5,6,7,8])

# integer indexing
print("Integer indexing")
print(f"data = {data}")
print(f"data[3] = {data[3]}")
print(f"data[0:2] = {data[0:2]}")
print(f"data[-2] = {data[-2]}")
print(f"data[::-4] = {data[::-4]}")

# fancy indexing
print("\nFancy indexing")
print(f"data[[1,6,3]] = {data[[1,6,3]]}")

# boolean indexing
print("\nBoolean indexing")
print(f"data[data>5] = {data[data>5]}")
```

```
# 2D example

data = np.array([[1, 2, 3, 4],[5, 6, 7, 8],[9, 10, 11, 12]])

# integer indexing
print("Integer indexing")
print(f"data[1] = {data[1]}")
print(f"data[:, 1] = {data[:, 1]}")
print(f"data[1:3, 2:4] = {data[1:3, 2:4]}")

# fancy indexing
print("\nFancy indexing")
print(f"data[[0,2,1], [2,3,0]] = {data[[0,2,1], [2,3,0]]}")

# boolean indexing
print("\nBoolean indexing")
print(f"data[data>10] = {data[data>10]}")
```

2.2.6 Array reshaping

Sometimes, you need to change the dimension of an array. One of the most common need is to transposing the matrix during the dot product. Switching the dimensions of a NumPy array is also quite common in more advanced cases.

```
import numpy as np

data = np.array([1,2,3,4,5,6,7,8,9,10,11,12])
print(f"data = \n{data}\n")

print(f"data.reshape(3,4) = \n{data.reshape(3,4)}\n")

print(f"data.reshape(4,3) = \n{data.reshape(4,3)}")
```

2.2.7 I/O with NumPy

Numpy provides functions for reading from/writing to files. Both ASCII and binary formats are supported with the CSV and npy/npz formats.

CSV

The `numpy.loadtxt()` and `numpy.savetxt()` functions can be used. They save in a regular column layout and can deal with different delimiters, column titles and numerical representations.

```
a = np.array([1, 2, 3, 4])
np.savetxt("my_array.csv", a)
b = np.loadtxt("my_array.csv")

print(a == b) # [ True  True  True  True]
```

Binary

The npy format is a binary format used to dump arrays of any shape. Several arrays can be saved into a single npz file, which is simply a zipped collection of different npy files. All the arrays to be saved into a npz file can be passed as kwargs to the `numpy.savez()` function. The data can then be recovered using the `numpy.load()` method, which returns a dictionary-like object in which each key points to one of the arrays.

```
a = np.array([1, 2, 3, 4])
b = np.array([5, 6, 7, 8])

np.savez("my_arrays.npz", array_1=a, array_2=b)
data = np.load("my_arrays.npz")

print(data['array_1'] == a) # [ True  True  True  True]
print(data['array_2'] == b) # [ True  True  True  True]
```

2.2.8 Random numbers

The module `numpy.random` provides several functions for constructing random arrays

`random()` : uniform random numbers

`normal()` : normal distribution

`choice()` : random sample from given array

...

```
import numpy as np

print(np.random.random((2,2)),'\n')

print(np.random.choice(np.arange(4), 10))
```

2.3 Pandas

Pandas is a Python package that provides high-performance and easy to use data structures and data analysis tools. Built on NumPy arrays, Pandas is particularly well suited to analyze tabular and time series data. Although NumPy could in principle deal with structured arrays (arrays with mixed data types), it is not efficient.

The core data structures of Pandas are Series and Dataframes.

A Pand`as ser`ies is a one-dimensional NumPy array with an index which we could use to access the data `- A dataf`rame consist of a table of values with labels for each row and column. A dataframe can combine multiple data types, such as numbers and text, but the data in each column is of the same type.

Each column of a dataframe is a series object - a dataframe is thus a colle tion of series.

2.3.1 Tidy vs untidy data

Most tabular data is either in a tidy format or a untidy format (some people refer them as the long format or the wide format).

In untidy (wide) format, each row represents an observation consisting of multiple variables and each variable has its own column. This is intuitive and easy for us to understand and make comparisons across different variables, calculate statistics,* etc*.

In tidy (long) formt , *i.*,e. column-oriented format, each row represents only one variable of the observation, and can be considered “computer readable”.

When it comes to data analysis using Pandas, the tidy format is recommended:

Each column can be stored as a vector and this not only saves memory but also allows for vectorized calculations which are much faster.

It's easier to filter, group, join and aggregate the data.

The name "tidy data" comes from Wickham's paper (2014) which describes the ideas in great detail.

2.3.2 Data analysis workflow

Pandas is a powerful tool for many steps of a data analysis pipeline:

Downloading and reading in datasets

Initial exploration of data

Pre-processing and cleaning data

renaming, reshaping, reordering, type conversion

handling duplicate/missing/invalid data

Analysis

To explore some of the capabilities, we start with an example dataset containing the passenger list from the Titanic, which is often used in Kaggle competitions and data science tutorials. First step is to load Pandas and download the dataset into a dataframe.

```
import pandas as pd

url = "https://raw.githubusercontent.com/pandas-dev/pandas/master/doc/data/titanic.csv"

# set the index to the "Name" column
titanic = pd.read_csv(url, index_col="Name")
```

Pandas also understands multiple other formats, for example `read_excel()`, `read_hdf()`, `read_json()`, *etc.* (and corresponding methods to write to file: `to_csv()`, `to_excel()`, `to_hdf()`, `to_json()`, ...)

We can now view the dataframe to get an idea of what it contains and print some summary statistics of its numerical data:

```
# print the first 5 lines of the dataframe
print(titanic.head())

# print some information about the columns
print(titanic.info())

# print summary statistics for each column
print(titanic.describe())
```

Now we have information on passenger names, survival (0 or 1), age, ticket fare, number of siblings/spouses, etc. With the summary statistics we see that the average age is 29.7 years, maximum ticket price is 512 USD, 38% of passengers survived, *etc.*

Unlike a NumPy array, a dataframe can combine multiple data types, such as numbers and text, but the data in each column is of the same type. So we say a column is of type `int64` or of type `object`.

2.3.3 Indexing

Let's inspect one column of the dataframe:

```
titanic["Age"] # same as "titanic.Age"
```

The columns have names. Here's how to get them:

```
titanic.columns
```

However, the rows also have names! This is what Pandas calls the `index`:

```
titanic.index
```

We saw above how to select a single column, but there are many ways of selecting (and setting) single or multiple rows, columns and elements. We can refer to columns and rows either by number or by their name:

```
print(titanic.loc["Lam, Mr. Ali","Age"])              # select single value by row and column
print(titanic.loc["Lam, Mr. Ali","Survived":"Age"])  # slice the dataframe by row and column *names*
print(titanic.iloc[692,3:6])                          # same slice as above by row and column *numbers*

print(titanic.at["Lam, Mr. Ali", "Age"])              # select single value by row and column *name* (fast)

titanic.at["Lam, Mr. Ali", "Age"] = 44                # set single value by row and column *name* (fast)
print(titanic.iat[692,4])                             # select same value by row and column *number* (fast)

# print(titanic["somecolumns"] = "somevalue")         # set a whole column
```

2.3.4 Missing/invalid data

What if your dataset has missing data? Pandas uses the value np.nan to represent missing data, and by default does not include it in any computations. We can find missing values, drop them from our dataframe, replace them with any value we like or do forward or backward filling.

```
titanic.isna()                    # returns boolean mask of NaN values
```

```
print(titanic.dropna())                  # drop missing values
print(titanic.dropna(how="any"))         # or how="all"
print(titanic.dropna(subset=["Cabin"]))  # only drop NaNs from one column
print(titanic.fillna(0))                 # replace NaNs with zero
```

```
print(titanic.ffill())    # forward-fill NaNs
print(titanic.bfill())    # backward-fill NaNs
```

2.3.5 Groupby

`groupby()` is a powerful method which splits a dataframe and aggregates data in groups. To see what's possible, let's test the old saying "Women and children first". We start by creating a new column *Child* to indicate whether a passenger was a child or not, based on the existing *Age* column. For this example, let's assume that you are a child when you are younger than 12 years:

```
titanic["Child"] = titanic["Age"] < 12
titanic["Child"]
```

Now we can test the saying by grouping the data on Sex and then creating further sub-groups based on Child:

```
titanic.groupby(["Sex", "Child"])["Survived"].mean()
```

Here we chose to summarize the data by its mean, but many other common statistical functions are available as dataframe methods, like `std()`, `min()`, `max()`, `cumsum()`, `median()`, `skew()`, `var()`, *etc.*

The workflow of `groupby()` can be divided into three general steps:

Splitting: Partition the data into different groups based on some criterion.

Applying: Do some caclulation within each group. Different kinds of calulations might be aggregation, transformation, filtration.

Combining: Put the results back together into a single object.

(Image source from lecture An Introduction to Earth and Environmental Data Science

2.4 Scipy

SciPy is a library that builds on top of NumPy. It contains a lot of interfaces to battle-tested numerical routines written in Fortran or C, as well as Python implementations of many common algorithms.

Briefly, SciPy contains functionality for

Special functions (Bessel, Gamma, *etc.*)

Numerical integration

Optimization

Interpolation

Fast Fourier Transform (FFT)

Signal processing

Linear algebra (more complete than in NumPy)

Sparse matrices

Statistics

More I/O routine, *e.g.* Matrix Market format for sparse matrices, MATLAB files wer-law to a vector.tor.t), *etc.*s.
Many of these are not written specifically for SciPy, but use the best available open source C or Fortran libraries. Thus, you get the best of Python and the best of compiled languages.

Most functions are documented very well from a scientific standpoint: you aren't just using some unknown function, but have a full scientific description and citation to the method and implementation.

Let us look more closely into one out of the countless useful functions available in SciPy. `curve_fit()` is a non-linear least squares fitting function. NumPy has least-squares fitting via the `np.linalg.lstsq()` function, but we need to go to SciPy to find non-linear curve fitting. This example fits a power-law to a vector.

```
import numpy as np
from scipy.optimize import curve_fit

def powerlaw(x, A, s):
    return A * np.power(x, s)

# data
Y = np.array([9115, 8368, 7711, 5480, 3492, 3376, 2884, 2792, 2703, 2701])
X = np.arange(Y.shape[0]) + 1.0

# initial guess for variables
p0 = [100, -1]
# fit data
params, cov = curve_fit(f=powerlaw, xdata=X, ydata=Y, p0=p0, bounds=(-np.inf, np.inf))

print("A =", params[0], "+/-", cov[0,0]**0.5)
print("s =", params[1], "+/-", cov[1,1]**0.5)

# optionally plot
import matplotlib.pyplot as plt
plt.plot(X,Y)
plt.plot(X, powerlaw(X, params[0], params[1]))
plt.show()
```

2.5 Exercises

2.5.1 Working effectively with dataframes

Recall the `curve_fit()` method from SciPy discussed above, and imagine that we want to fit powerlaws to every row in a large dataframe. How can this be done effectively?

First define the `powerlaw()` function and another function for fitting a row of numbers:

```
import numpy as np
import pandas as pd
from scipy.optimize import curve_fit

def powerlaw(x, A, s):
    return A * np.power(x, s)

def fit_powerlaw(row):
    X = np.arange(row.shape[0]) + 1.0
    params, cov = curve_fit(f=powerlaw, xdata=X, ydata=row, p0=[100, -1], bounds=(-np.inf, np.inf))
    return params[1]
```

Next load a dataset with multiple rows similar to the one used in the example above:

```
df = pd.read_csv("https://raw.githubusercontent.com/ENCCS/hpda-python/main/content/data/results.csv")
# print first few rows
df.head()
```

Now consider these four different ways of fitting a powerlaw to each row of the dataframe:

```
# 1. Loop

powers = []
for row_indx in range(df.shape[0]):
    row = df.iloc[row_indx,1:]
    p = fit_powerlaw(row)
    powers.append(p)
```

```
# 2. `iterrows()

powers = []
for row_indx,row in df.iterrows():
    p = fit_powerlaw(row[1:])
    powers.append(p)
```

```
# 3. `apply()

powers = df.iloc[:,1:].apply(fit_powerlaw, axis=1)
```

```
# 4. `apply()` with `raw=True`

# raw=True passes numpy ndarrays instead of series to fit_powerlaw
powers = df.iloc[:,1:].apply(fit_powerlaw, axis=1, raw=True)
```

Which one do you think is most efficient? You can measure the execution time by adding `%%timeit` to the first line of a Jupyter code cell. More on timing and profiling in a later episode.

▶ Solution

2.5.2 Further analysis of Titanic passenger list dataset

Consider the titanic dataset.

If you haven't done so already, load it into a dataframe before the exercises:

```
import pandas as pd; url = "https://raw.githubusercontent.com/pandas-dev/pandas/master/doc/data/titanic.csv"; titanic = pd.read_csv(url, index_col="
```

Compute the mean age of the first 10 passengers by slicing and the `mean` method

Using boolean indexing, compute the survival rate (mean of "Survived" values) among passengers over and under the average age. Now investigate the family size of the passengers (*i.e.* the "SibSp" column):

What different family sizes exist in the passenger list?

Hint: try the `unique()` method

What are the names of the people in the largest family group?

(Advanced) Create histograms showing the distribution of family sizes for passengers split by the fare, i.e. one group of high-fare passengers (where the fare is above average) and one for low-fare passengers

Hint: instead of an existing column name, you can give a lambda function as a parameter to `hist` to compute a value on the fly. For example `lambda x: "Poor" if titanic["Fare"].loc[x] < titanic["Fare"].mean() else "Rich"`).

▶ Solution

2.6 Keypoints

NumPy provides a static array data structure, fast mathematical operations for arrays and tools for linear algebra and random numbers

Pandas dataframes are a good data structure for tabular data

Dataframes allow both simple and advanced analysis in very compact form

SciPy contains a lot of interfaces to battle-tested numerical routines

2.7 Episode Quizzes

2.7.1 Choice questions

Why are Python lists inefficient for numerical computations?

A) They store elements as generic objects with dynamic typing

B) They are statically typed

C) They don't support indexing

D) They don't support loops

What is the main advantage of NumPy arrays (ndarray) over Python lists for numerical tasks?

A) They can hold multiple data types

B) They automatically parallelize loops

C) They store data in a compact, contiguous block of memory

D) They have larger memory overhead

What is "vectorization" in the context of NumPy?

A) A way to convert lists to dictionaries

B) A process of compiling Python code

C) A plotting technique

D) Replacing explicit loops with whole-array operations

How does a pandas DataFrame differ from a NumPy array?

A) DataFrames are slower and less powerful

B) DataFrames support heterogeneous data types and labeled axes

C) Arrays use less memory

D) DataFrames cannot be indexed

What does `scipy.optimize.curve_fit()` do?

A) Performs numerical integration

B) Fits data to a model function

C) Solves a linear system

D) Computes a histogram

2.7.2 Coding questions

Generate a 1D NumPy array of 1 million random floats. Compute the square root of each element using:

a) a Python for loop

b) NumPy's vectorized np.sqrt

Load a CSV file of weather data (*e.g.*, temperature, humidity, wind).

a) filter rows where temperature > 30°C

b) compute the average humidity for each month using `groupby`

Create a random 100×100 matrix A and a vector b.

a) use `scipy.linalg.solve` to solve the system $Ax = b$

b) verify the solution by checking the residual norm

Simulate a DataFrame with missing values in numerical columns.

a) fill missing values with the column mean (using NumPy)

b) compute basic statistics before and after imputation

Generate noisy data for a quadratic function $y = ax^2 + bx + c$

a) use `scipy.optimize.curve_fit` to fit the data and recover the original parameters

b) plot the original *vs* fitted curve

Quick Reference

Instructor's guide

Why we teach this lesson

Intended learning outcomes

Timing

Preparing exercises

e.g. what to do the day before to set up common repositories.

Other practical aspects

Interesting questions you might get

Typical pitfalls

Who is the course for?

About the course

See also

Credits